Snowpark Container Services (SPCS) — Developer Cheatsheet

Deploy and manage containerized workloads in Snowflake using the `snow spcs` CLI. Available on AWS, Azure, and GCP commercial regions.

> [!IMPORTANT] Community cheatsheet — not official Snowflake documentation. For the authoritative reference, see Snowpark Container Services docs.

Table of Contents

Prerequisites

Services cannot be created as `ACCOUNTADMIN` — a custom role with specific grants is required. Run the setup script to create the required role, database, schema, and warehouse:

```
# run in terminal — as ACCOUNTADMIN
curl https://raw.githubusercontent.com/Snowflake-Labs/sf-cheatsheets/main/samples/spcs_setup.sql \
  | snow sql --stdin
```

This creates:

Object	Name	Purpose
Role	`cheatsheets_spcs_demo_role`	

Object	Name	Purpose
		Create and manage SPCS resources
Database	`CHEATSHEETS_DB`	Holds services and image repositories
Schema	`DATA_SCHEMA`	Scopes image repositories and services
Warehouse	`cheatsheets_spcs_wh_s`	Used by service containers for SQL queries

Compute Pool

Instance families available:

Family	CPU/GPU	Notes
`CPU_X64_XS` / `S` / `M` / `L`	CPU	Standard CPU workloads
`HIGHMEM_X64_S` / `M` / `L`	CPU + high memory	Memory-intensive workloads
`GPU_NV_S` / `M` / `L`	NVIDIA GPU	ML inference and training

Create a compute pool with common options in one block:

```
# run in terminal
snow spcs compute-pool create my_pool \
  --family CPU_X64_XS \    # required
  --min-nodes 1 \           # scale-down floor (default: 1)
  --max-nodes 3 \           # scale-out ceiling (default: 1)
  --auto-suspend-secs 300 \ # idle seconds before suspend
(default: 3600)
  --auto-resume \           # auto-start on service/job request
(default: true)
  --if-not-exists
```

Lifecycle commands:

```
# run in terminal
snow spcs compute-pool list                         # list all
pools
snow spcs compute-pool list --like 'my_%'           # filter by
name
snow spcs compute-pool describe my_pool             # full details
snow spcs compute-pool status my_pool               # runtime
```

```
status
snow spcs compute-pool suspend my_pool              # stop pool
snow spcs compute-pool resume my_pool               # start pool
snow spcs compute-pool set my_pool --auto-suspend-secs 120  #
update property
snow spcs compute-pool drop my_pool                 # remove pool
```

Image Registry

```
# run in terminal
snow spcs image-registry login    # docker login (requires Docker
on local system)
snow spcs image-registry token    # get auth token for current
user
snow spcs image-registry url      # get registry URL for this
account
```

Image Repository

```
# run in terminal
snow spcs image-repository create my_repo \
  --database CHEATSHEETS_DB \
  --schema DATA_SCHEMA \
  --role cheatsheets_spcs_demo_role \
  --if-not-exists
```

Push an image to the repository:

```
# run in terminal
REPO=$(snow spcs image-repository url my_repo \
  --database CHEATSHEETS_DB --schema DATA_SCHEMA \
  --role cheatsheets_spcs_demo_role)

docker pull --platform=linux/amd64 nginx
docker tag nginx "$REPO/nginx"
docker push "$REPO/nginx"
```

List and inspect images:

```
# run in terminal
snow spcs image-repository list \
  --database CHEATSHEETS_DB --schema DATA_SCHEMA    # list repos

snow spcs image-repository list-images my_repo \
  --database CHEATSHEETS_DB --schema DATA_SCHEMA    # list images

snow spcs image-repository list-tags my_repo \
  --image-name=/CHEATSHEETS_DB/DATA_SCHEMA/my_repo/nginx \
  --database CHEATSHEETS_DB --schema DATA_SCHEMA    # list tags
```

```
snow spcs image-repository drop my_repo \
  --database CHEATSHEETS_DB --schema DATA_SCHEMA    # remove repo
```

> [!TIP] Use `--format json | jq -r '.[0].image'` to extract the fully qualified image name for use in a service spec.

Services: Long-running

A long-running service stays up until explicitly stopped. Snowflake restarts crashed containers.

Service spec (save as `service-spec.yaml`):

```yaml
spec:
  containers:
    - name: nginx
      image: /CHEATSHEETS_DB/DATA_SCHEMA/my_repo/nginx:latest
      readinessProbe:
        port: 80
        path: /
  endpoints:
    - name: nginx
      port: 80
      public: true
```

Create a service:

```
# run in terminal
snow spcs service create my_svc \
  --compute-pool my_pool \
  --spec-path service-spec.yaml \
  --min-instances 1 \         # minimum replicas (default: 1)
  --max-instances 3 \         # maximum replicas
  --query-warehouse cheatsheets_spcs_wh_s \  # warehouse for
container SQL
  --database CHEATSHEETS_DB \
  --schema DATA_SCHEMA \
  --role cheatsheets_spcs_demo_role \
  --if-not-exists
```

Deploy from a `snowflake.yml` project definition:

```yaml
# snowflake.yml
definition_version: 2
entities:
  my_svc:
    type: service
    compute_pool: my_pool
    spec_file: service-spec.yaml
    min_instances: 1
```

```
    max_instances: 3
    query_warehouse: cheatsheets_spcs_wh_s

# run in terminal — from the directory containing snowflake.yml
snow spcs service deploy
```

Lifecycle and inspection commands:

```
# run in terminal
snow spcs service status my_svc        # runtime status (wait for
READY)
snow spcs service describe my_svc      # full service details
snow spcs service list                 # list all services
snow spcs service list --like 'my_%'   # filter by name
snow spcs service list-endpoints my_svc      # public/internal
endpoints
snow spcs service list-instances my_svc     # instances and their
status
snow spcs service list-containers my_svc    # containers in each
instance
snow spcs service list-roles my_svc         # service roles for
endpoint access
snow spcs service logs my_svc \
  --container-name nginx --instance-id 0   # container logs
snow spcs service events my_svc            # event history
snow spcs service metrics my_svc           # performance metrics
snow spcs service suspend my_svc           # stop accepting
requests
snow spcs service resume my_svc            # resume accepting
requests
snow spcs service set my_svc \
  --min-instances 2 --max-instances 5      # update scaling
snow spcs service unset my_svc --min-instances  # reset to default
snow spcs service upgrade my_svc \
  --spec-path service-spec.yaml            # deploy new spec
snow spcs service drop my_svc             # remove service
```

Cleanup all resources:

```
# run in terminal
curl https://raw.githubusercontent.com/Snowflake-Labs/sf-
cheatsheets/main/samples/spcs_cleanup.sql \
  | snow sql --stdin
```

Services: Job (execute-job)

A job service runs a container workload that terminates when all containers exit — like a stored procedure but containerized. Snowflake cleans up resources automatically.

```
# run in terminal — synchronous (waits for completion)
snow spcs service execute-job my_job \
  --compute-pool my_pool \
  --spec-path job-spec.yaml \
  --database CHEATSHEETS_DB \
  --schema DATA_SCHEMA \
  --role cheatsheets_spcs_demo_role

# asynchronous (returns immediately, job runs in background)
snow spcs service execute-job my_job \
  --compute-pool my_pool \
  --spec-path job-spec.yaml \
  --async \
  --database CHEATSHEETS_DB --schema DATA_SCHEMA
```

After the job completes, use `snow spcs service describe my_job` to inspect results. Job metadata is retained for 30 days.

Permissions

Minimum grants required for a role to create and manage SPCS resources:

```
-- Run as ACCOUNTADMIN
GRANT CREATE COMPUTE POOL ON ACCOUNT TO ROLE spcs_role;
GRANT BIND SERVICE ENDPOINT ON ACCOUNT TO ROLE spcs_role;

-- On the target database and schema
GRANT USAGE ON DATABASE mydb TO ROLE spcs_role;
GRANT USAGE, CREATE SERVICE ON SCHEMA mydb.myschema TO ROLE
        spcs_role;

-- On supporting objects
GRANT USAGE ON WAREHOUSE mywh TO ROLE spcs_role;
GRANT USAGE ON COMPUTE POOL my_pool TO ROLE spcs_role;
GRANT OPERATE, MONITOR ON COMPUTE POOL my_pool TO ROLE spcs_role;

-- On the image repository (read access to pull images)
GRANT READ ON IMAGE REPOSITORY mydb.myschema.my_repo TO ROLE
        spcs_role;
```

Tips & Gotchas

- **No ACCOUNTADMIN for services.** Service creation requires a custom role — attempting to use ACCOUNTADMIN fails with a permission error. Run `spcs_setup.sql` first.
- **Long-running vs job services.** Use `snow spcs service create` for persistent workloads (web servers, APIs). Use `snow spcs service execute-job` for batch/ETL work that terminates. Jobs clean up compute resources automatically when done.

- **Image name must be fully qualified.** `list-tags --image-name` requires the full path `/DB/SCHEMA/REPO/image` — use `list-images` to get the exact string.
- **READY status takes time.** After `service create`, the pool needs to provision nodes. Poll with `service status` and wait for `RUNNING` before testing endpoints.
- **Compute pool cache drops on suspend.** Like warehouses, suspending the pool clears the local node cache. Avoid suspending pools with latency-sensitive services unless cost savings outweigh cold-start time.

References

- Snowpark Container Services overview
- snow spcs commands reference
- Service specification reference
- Working with image registries and repositories
- Managing services
- SPCS tutorials